

OOP Project Design Report

Kyrylo Pryiomyshev

Project: RoomSizer - Wallpaper Calculator Application

13. November 2025

Introduction

RoomSizer is a wallpaper calculator that determines how many wallpaper rolls are needed for a room, accounting for room dimensions, openings (windows/doors), pattern matching, and waste factors. The application provides three interfaces: interactive CLI, non-interactive CLI, and Flask web API. Main features include strip-based calculation algorithm, comprehensive input validation, and robust logging.

Class Design

1. Domain Layer (`domain.py`): `OpeningKind` (Enum), `Opening` (value object), `Room` (entity), `WastePolicy` (value object), `StripBasedRollsCalculator` (algorithm implementation), `Wallpaper` (facade).

2. Ports Layer (`ports.py`): `AbstractOpening`, `AbstractRoom`, `AbstractWastePolicy`, `AbstractRollsCalculator` (all abstract base classes defining interfaces).

3. Adapters: CLI layer with `UserCancelled` exception, Flask web application.

4. Relationships: `Room` composes `Opening` objects (0-many). `StripBasedRollsCalculator` depends on `Room` and `WastePolicy`. `Wallpaper` aggregates `AbstractRollsCalculator` (Strategy pattern). All domain classes implement their corresponding abstract interfaces. CLI and web adapters depend on domain layer, which depends only on abstractions.

Application of OOP Principles

1. Encapsulation: `Room` uses private attributes (`__width`, `__length`, `__height`, `__openings`) with read-only `@property` accessors. `Opening` and `WastePolicy` are immutable (`@dataclass(frozen=True)`). `__slots__` used in

Room and StripBasedRollsCalculator for memory optimization and preventing dynamic attributes.

2. Inheritance: All domain classes implement abstract interfaces from ports layer using `@abstractmethod`.

3. Polymorphism: Wallpaper accepts any AbstractRollsCalculator implementation, enabling runtime strategy swapping. Concrete classes override abstract methods (e.g., `Opening.area()`, `StripBasedRollsCalculator.rolls_needed()`).

4. Abstraction: Ports layer defines abstract interfaces hiding implementation details. Wallpaper facade provides simplified API, hiding calculator complexity from clients.

Use of Design Patterns

1. Strategy Pattern: AbstractRollsCalculator interface with StripBasedRollsCalculator implementation. Wallpaper class is the context that can swap calculators at runtime. Enables adding new algorithms (e.g., ContinuousPatternCalculator) without modifying existing code.

2. Facade Pattern: Wallpaper class provides simplified interface to calculation subsystem, hiding complexity from CLI and web adapters.

3. Ports and Adapters (Hexagonal Architecture): Core domain logic separated from infrastructure. Ports define interfaces, domain implements them, adapters (CLI/web) depend on abstractions. Enables testing domain in isolation and adding interfaces without changing business logic.

4. Value Object Pattern: Opening and WastePolicy are immutable frozen dataclasses with value-based equality, providing thread safety and predictability.

SOLID Principles

1. Single Responsibility Principle: Each class has one responsibility (Opening manages opening data, Room manages dimensions, StripBasedRollsCalculator handles calculation algorithm, CLI handles user interaction).
2. Open/Closed Principle: New calculator algorithms can be added by implementing AbstractRollsCalculator without modifying existing classes. Abstract interfaces enable extension without modification.
3. Liskov Substitution Principle: Any AbstractRollsCalculator implementation can substitute another without breaking the system. Abstract base classes with type hints ensure behavioral consistency.
4. Interface Segregation Principle: Small, focused interfaces (AbstractOpening.area(), AbstractRollsCalculator.rolls_needed()) prevent unnecessary dependencies.
5. Dependency Inversion Principle: High-level modules (adapters) depend on abstractions (ports), not concrete implementations. Enables flexibility and testability.

Testing Strategy

Tests cover unit tests, boundary conditions, exception handling, and end-to-end workflows. MockInput class simulates user interaction for automated CLI testing. Pytest parametrization reduces duplication while increasing coverage.

Challenges and Design Decisions

1. Ports and Adapters Architecture: Chose hexagonal architecture to separate domain from infrastructure. Added initial complexity with abstract base classes, but enabled isolated testing and adding web interface without changing domain logic.

2. Immutability vs. Flexibility: Made Opening and WastePolicy immutable (frozen dataclasses) for thread safety, but kept Room mutable to allow incremental opening addition during CLI input. Pragmatic tradeoff favoring usability.

3. Validation Placement: Basic validation (positive dimensions) in domain constructors; higher-level validation (confirming unusual values) in CLI layer where user interaction is possible.

4. Strip-Based Algorithm: Implemented accurate strip-counting algorithm accounting for pattern matching, rather than simpler but less accurate area-based calculation. Complexity justified by accuracy.

5. Testing CLI: Created MockInput class to simulate user responses, requiring optional input source parameter in CLI functions. Enabled automated testing of interactive flows.

Conclusion

RoomSizer successfully demonstrates OOP principles, design patterns, and clean architecture.

With more time, I would add additional calculator strategies (continuous pattern, digital print), support non-rectangular rooms with composite pattern, expand web interface with modern frontend, implement persistence for room configurations, and add cost estimation features.